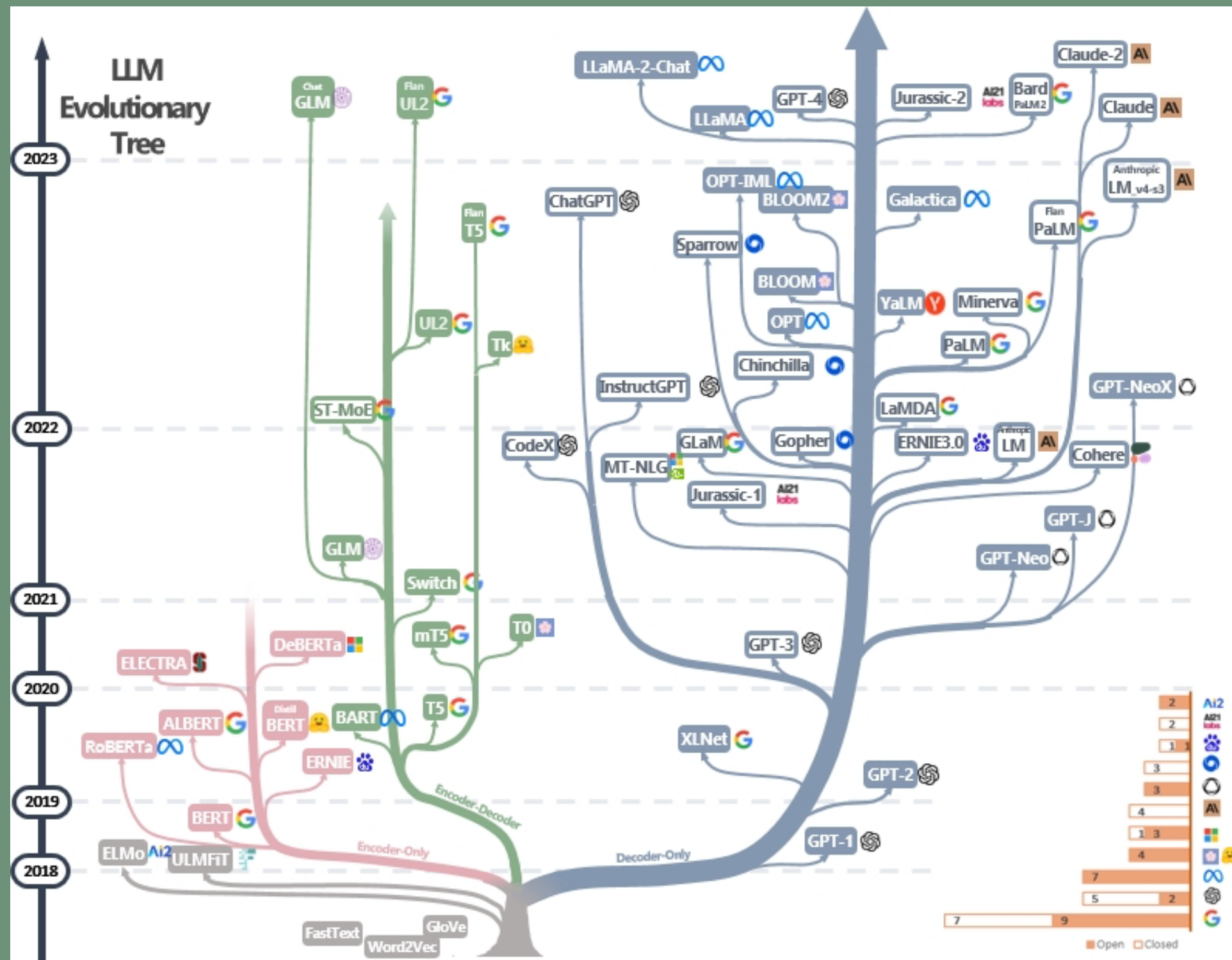


LLM Architectures

Lukas Galke Poech, AI509, Fall 2026

Recap from last session



Summary

- BERT-like encoder-only models are well-suited for text embedding out-of-the-box and a good basis task-specific fine-tuning, e.g., for text classification.
- T5-like encoder-decoder models are well-suited for sequence-to-sequence tasks, e.g., machine translation
- GPT-like decoder-only models are general purpose models, and with sufficient scale capable of in-context learning.

What is the key difference?

Both encoder-only and decoder-only models use Transformer blocks throughout. The only difference is whether we have a triangular attention mask.

Triangular attention mask = you can only interact with previous tokens, not future.

Encoder-decoder models use the original Transformer formulation (remember: this was for translation). The encoder part uses the full attention matrix, the decoder part has a diagonal attention mask.

Then, **cross-attention** i.e., decoder sends queries to encoder, to compare with encoder's keys, and receives back values.

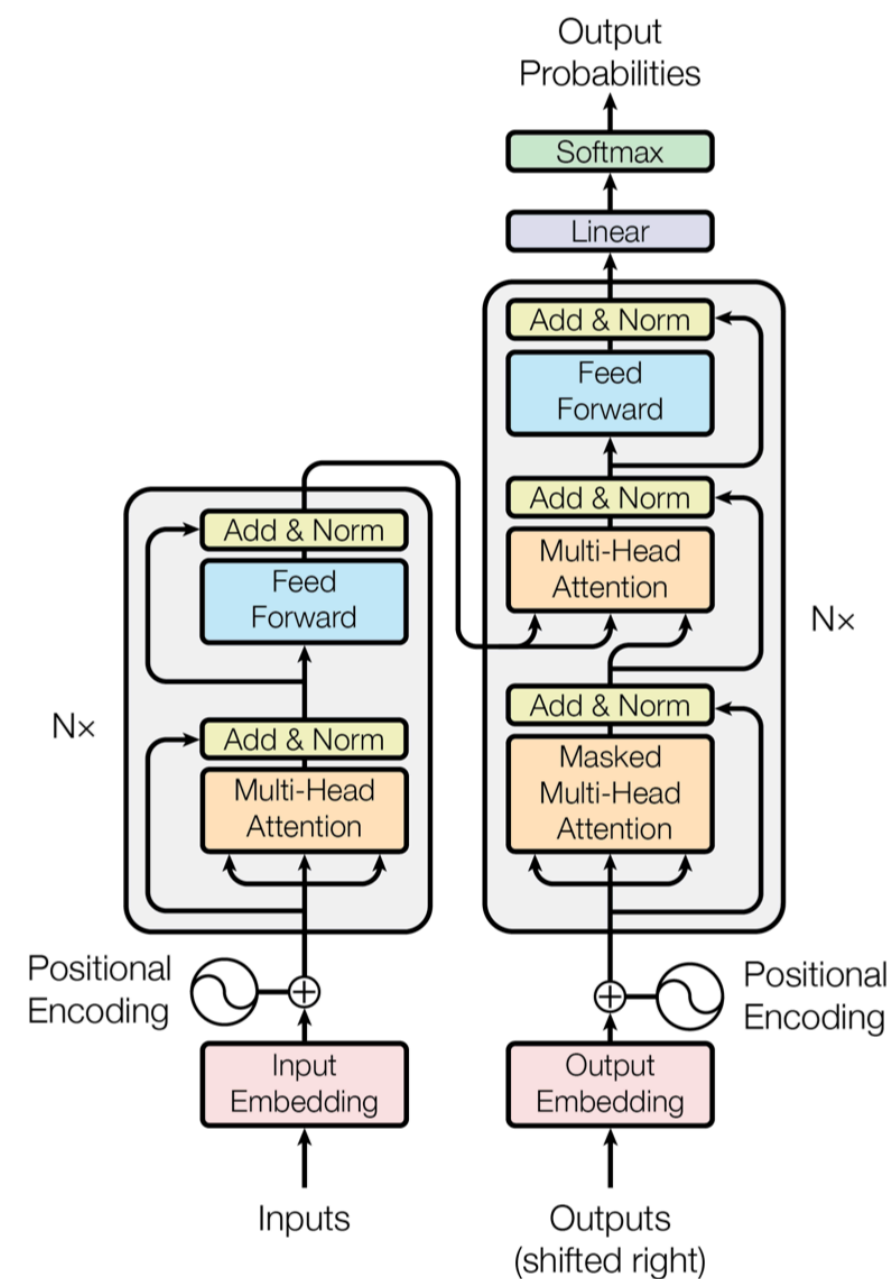


Figure 1: The Transformer - model architecture.

LLM training in a nutshell

1. Pre-training (language modeling on high amount of low quality data) → **base model**
2. Supervised Fine-tuning (medium amount of high quality data) → **instruction model**
3. Reinforcement Learning from Human feedback (small amount of preference data) → **chat model**

Keywords

- Mixture of Experts
- DeepSeek Sparse Attention
- Multi-Latent Attention
- Rotary Position Embeddings
- Multi-token Prediction
- Muon Optimizer
- RMSNorm
- Gated linear units

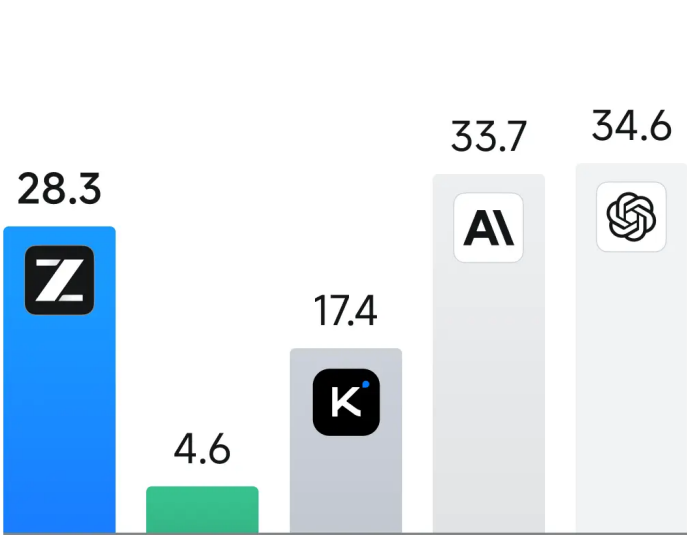
A look at GLM 5

LLM Performance Evaluation

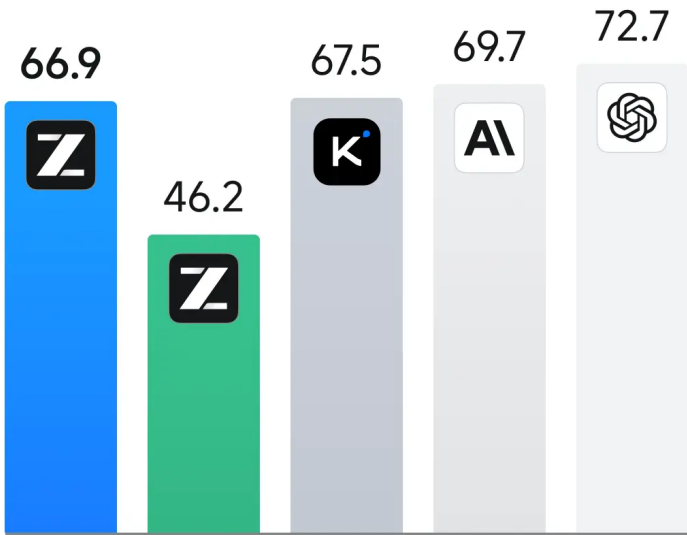


6 benchmarks: Terminal Bench 3.0, DeepSWE, Agents' Last Exam (CLI), AutomationBench, HLE w/ Tools, GDPVal-AA v2

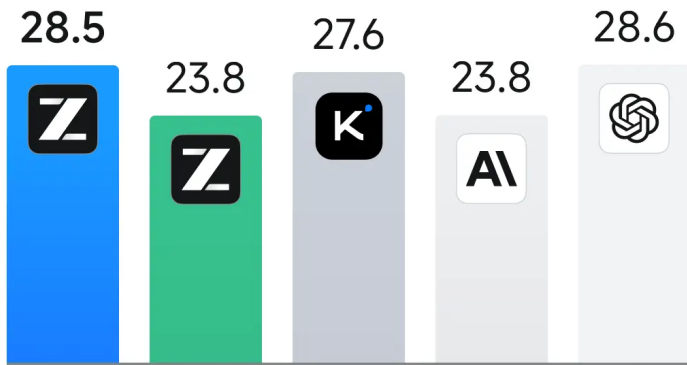
GLM-5.3 GLM-5.2 Kimi K3 Fable 5 GPT-5.6 Sol



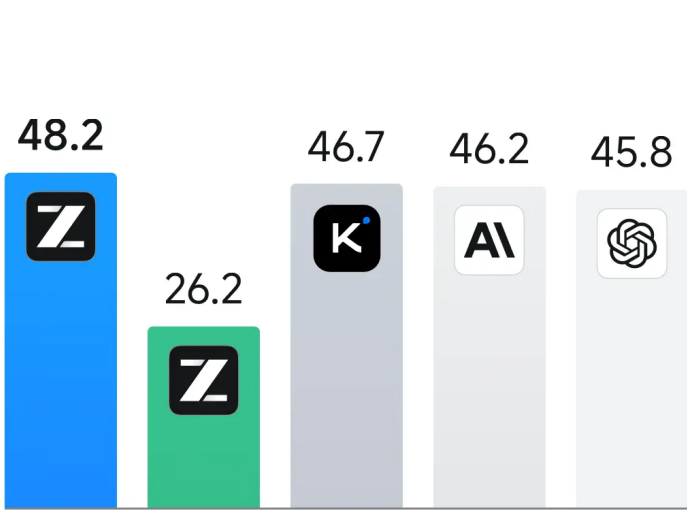
Terminal Bench 3.0



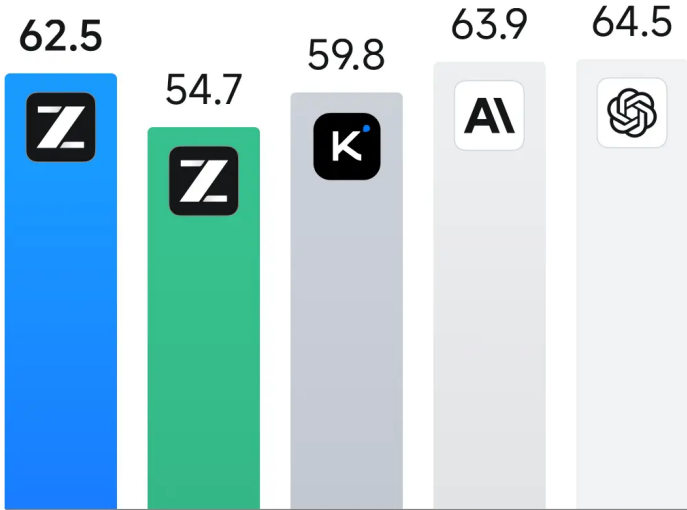
DeepSWE



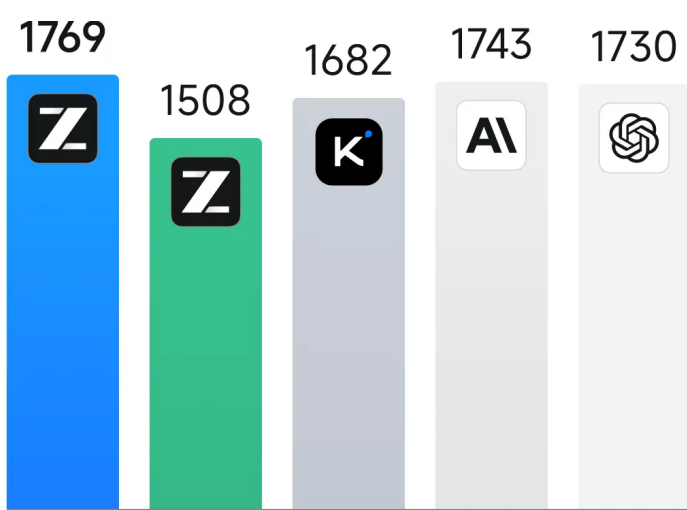
Agents' Last Exam



AutomationBench



HLE w/ Tools



GDPVal-AA v2

GLM 5.3, based on the GLM 5 architecture is one of the most powerful open-source models¹.

1. GLM-5 Team (2026). GLM-5: from Vibe Coding to Agentic Engineering. arXiv:2602.15763 ↩

Reading exercise

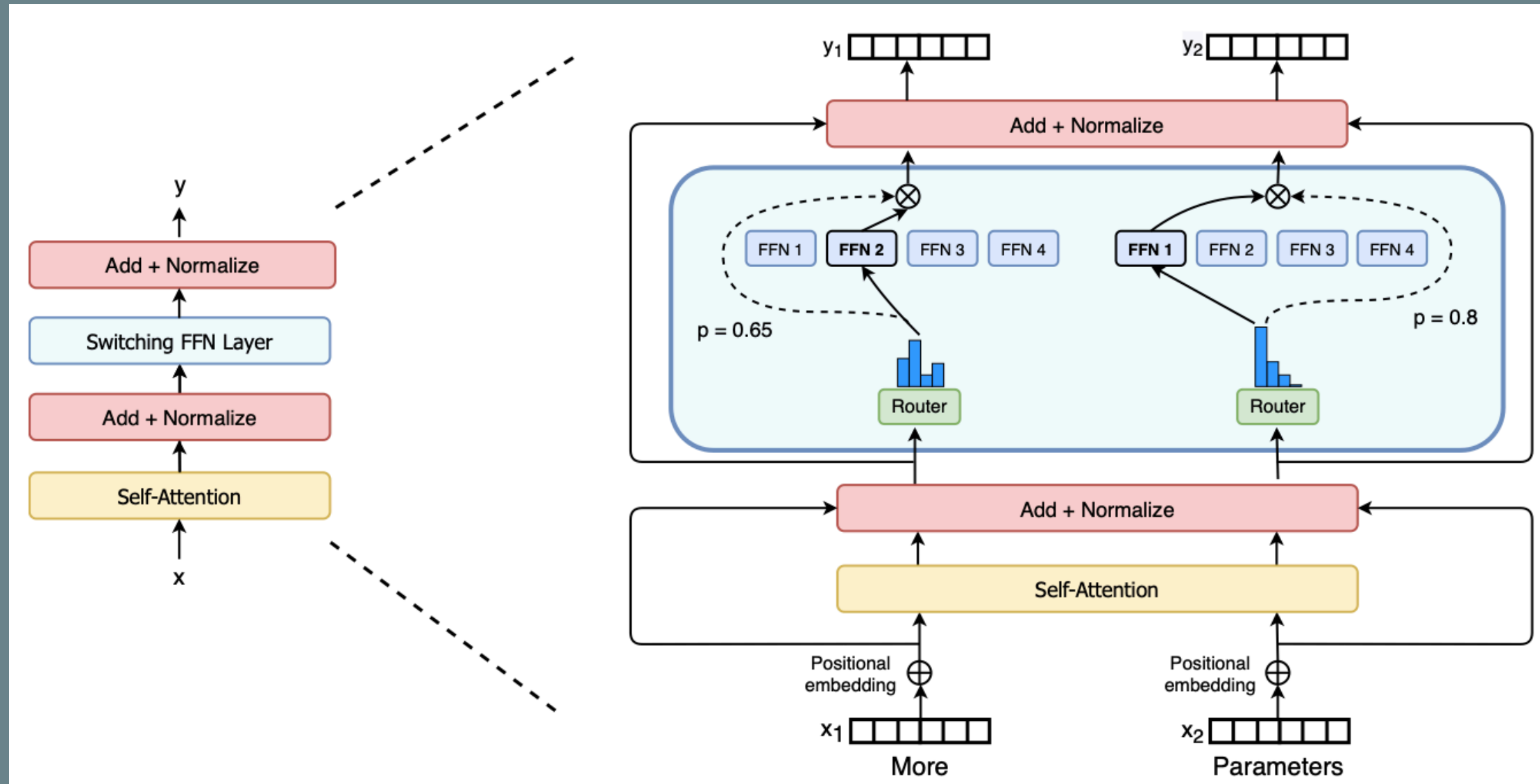
- Open the GLM 5 paper
- Read Section 2.1 Architecture
- Highlight the key components

Roadmap to GLM 5

- ☐ Mixture of Experts
- ☐ DeepSeek Sparse Attention
- ☐ Multi-head latent Attention
- ☐ Multi-token Prediction
- ☐ Muon Optimizer

Mixture of Experts

Switch Transformer¹



1. Fedus, Zoph, and Shazeer (JMLR 2022) ↩

Switch Transformer conceptually

1. Add a routing mechanism to determine which expert is the best fit.
2. Only the top-k *experts* (feedforward modules) get to process a given example
3. Outputs of these experts are aggregated according to routing scores

Switch Transformer in Math: TopK-Gate

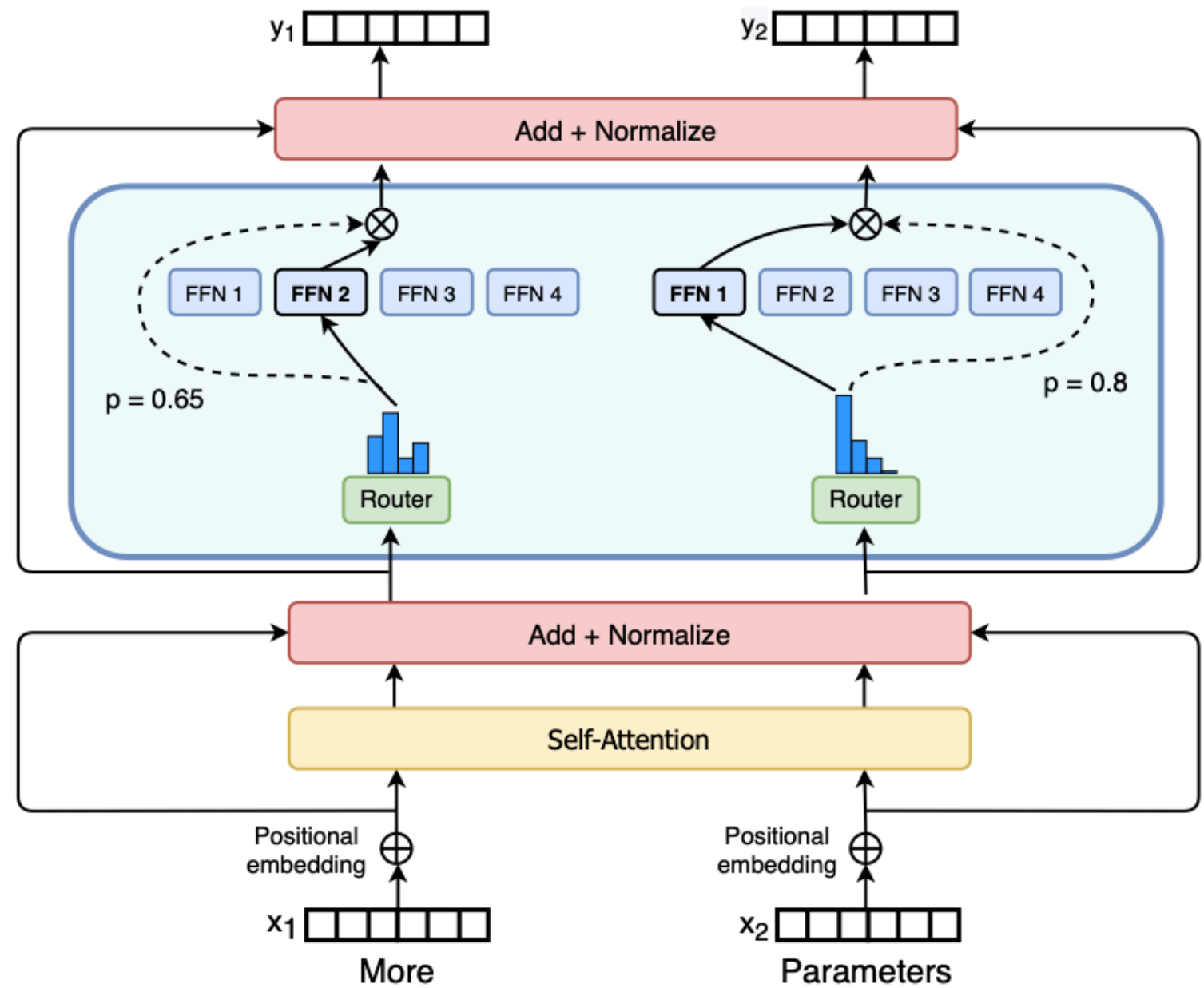
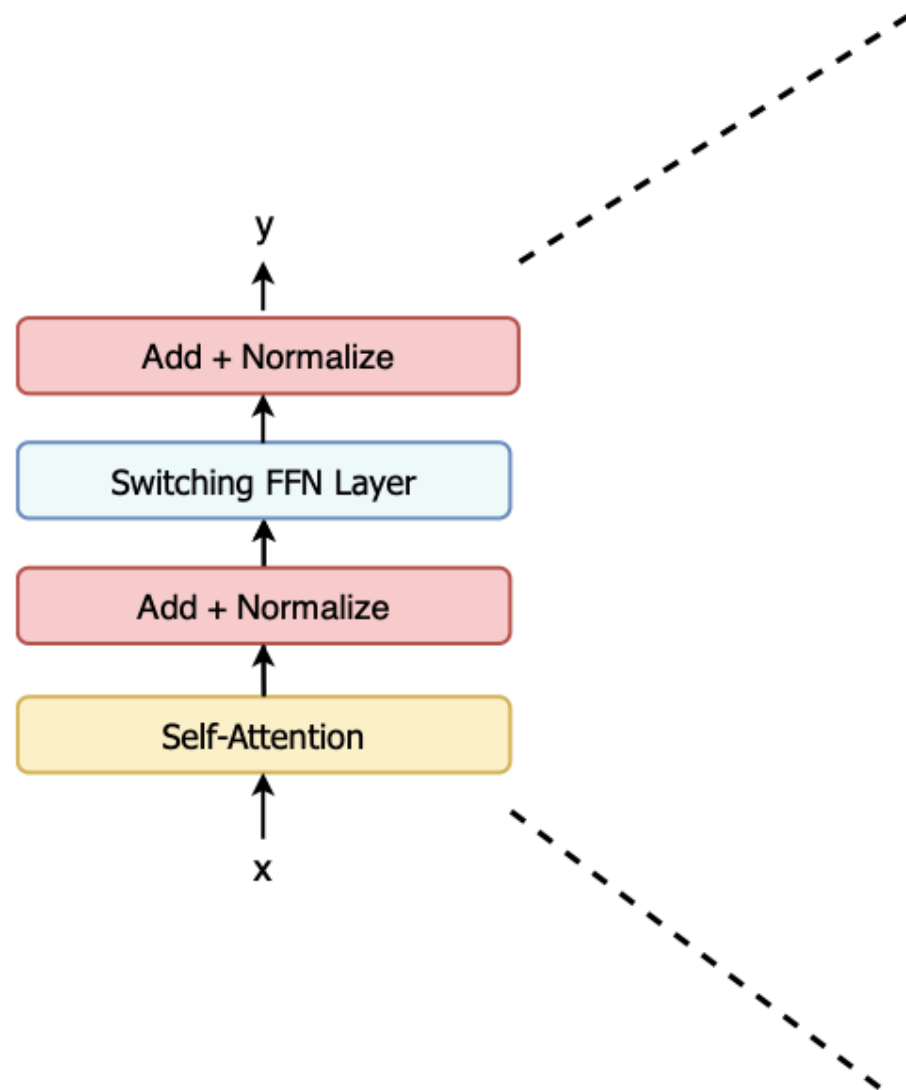
1. Add a routing variable W_R to produce logits $h(x) = W_r \dot{x}$
2. Estimate probability $p_i(x)$ that expert I is the best suited for token representation x :

$$p_i(x) = \frac{e^{h(x)_i}}{\sum_j^N e^{h(x)_j}} \text{ (a softmax!)}$$

3. When a set of experts T is selected, their outputs are aggregated by $y = \sum_{I \in T} p_i(x) E_i(x)$, where E_i is expert number i and $p_i(x)$ is the estimated probability that expert I is the best fit.

There's a bit more (but not much)

- Routing is done at each layer separately
- And for each token representation separately.
- But only within the feedforward blocks
- Add a regularization term to encourage balance between experts during training



Roadmap to GLM 5

- ☒ Mixture of Experts
- ☐ DeepSeek Sparse Attention
- ☐ Multi-head latent Attention
- ☐ Multi-token Prediction
- ☐ Muon Optimizer

**For DeepSeek Sparse
Attention, we need to
look at DeepSeek-V3.2¹**

1. DeepSeek-AI (2025). arXiv:2512.02556 ↩

DeepSeek-V3.2

1. **DeepSeek Sparse Attention (DSA)**
2. Scalable Reinforcement Learning Framework
3. Large-Scale Agentic Task Synthesis Pipeline

Prototype of DSA. The prototype of DSA primarily consists of two components: a lightning indexer and a fine-grained token selection mechanism.

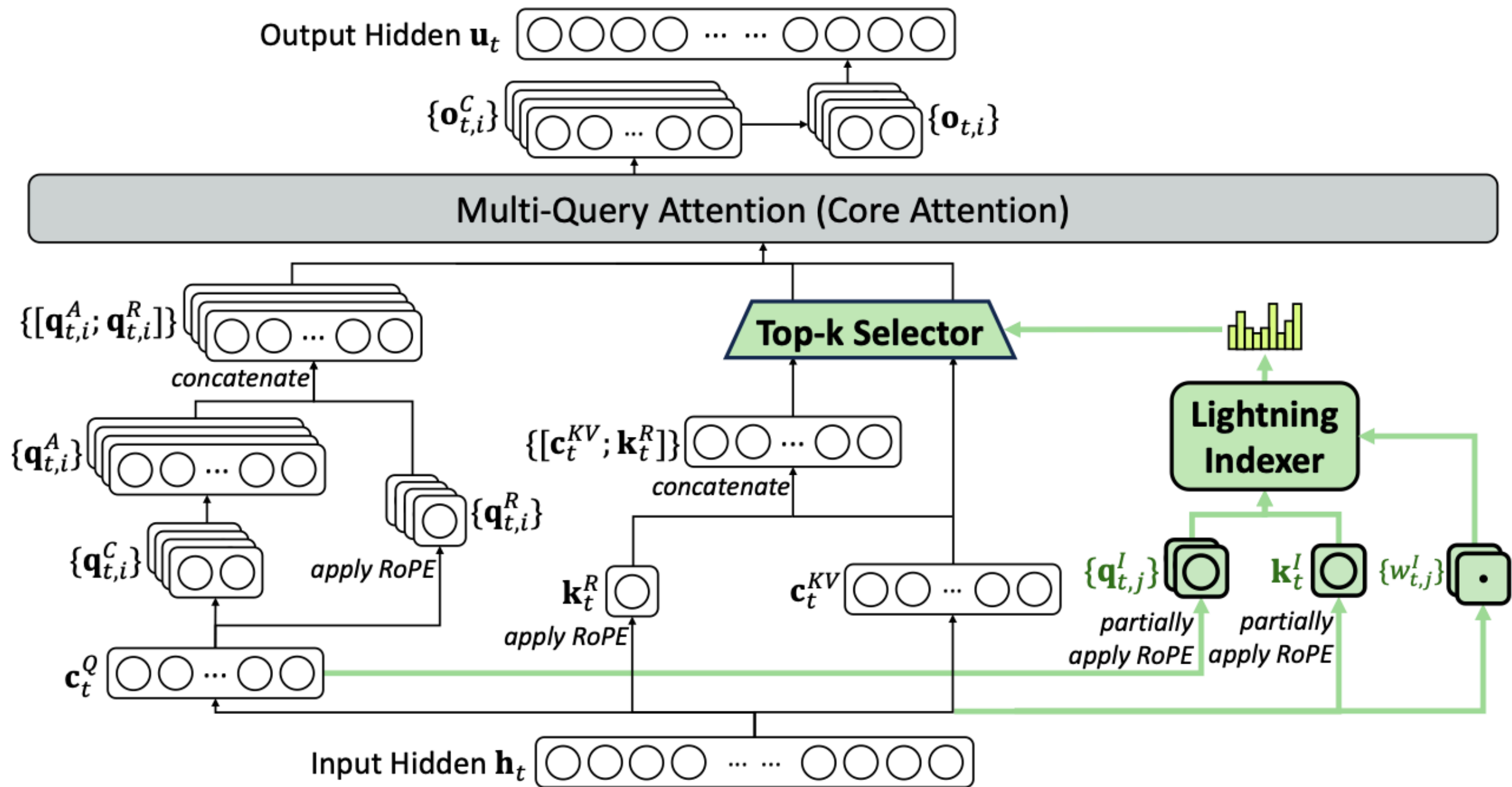
The **lightning indexer** computes the index score $I_{t,s}$ between the query token $\mathbf{h}_t \in \mathbb{R}^d$ and a preceding token $\mathbf{h}_s \in \mathbb{R}^d$, determining which tokens to be selected by the query token:

$$I_{t,s} = \sum_{j=1}^{H^I} w_{t,j}^I \cdot \text{ReLU}(\mathbf{q}_{t,j}^I \cdot \mathbf{k}_s^I), \quad (1)$$

where H^I denotes the number of indexer heads; $\mathbf{q}_{t,j}^I \in \mathbb{R}^{d^I}$ and $w_{t,j}^I \in \mathbb{R}$ are derived from the query token $\mathbf{h}_t$; and $\mathbf{k}_s^I \in \mathbb{R}^{d^I}$ is derived from the preceding token $\mathbf{h}_s$. We choose ReLU as the activation function for throughput consideration. Given that the lightning indexer has a small number of heads and can be implemented in FP8, its computational efficiency is remarkable.

Given the index scores $\{I_{t,s}\}$ for each query token $\mathbf{h}_t$, our **fine-grained token selection mechanism** retrieves only the key-value entries $\{\mathbf{c}_s\}$ corresponding to the top-k index scores. Then, the attention output $\mathbf{u}_t$ is computed by applying the attention mechanism between the query token $\mathbf{h}_t$ and the sparsely selected key-value entries $\{\mathbf{c}_s\}$:

$$\mathbf{u}_t = \text{Attn}(\mathbf{h}_t, \{\mathbf{c}_s \mid I_{t,s} \in \text{Top-k}(I_{t,:})\}). \quad (2)$$



Question: Why is DeepSeek Sparse Attention more efficient than standard multi-head attention?

Roadmap to GLM 5

- ☒ Mixture of Experts
- ☒ DeepSeek Sparse Attention
- ☐ Multi-latent Attention
 - ☐ Rotary Position Embeddings
- ☐ Multi-token Prediction
- ☐ Muon Optimizer

Rotary Position Embeddings

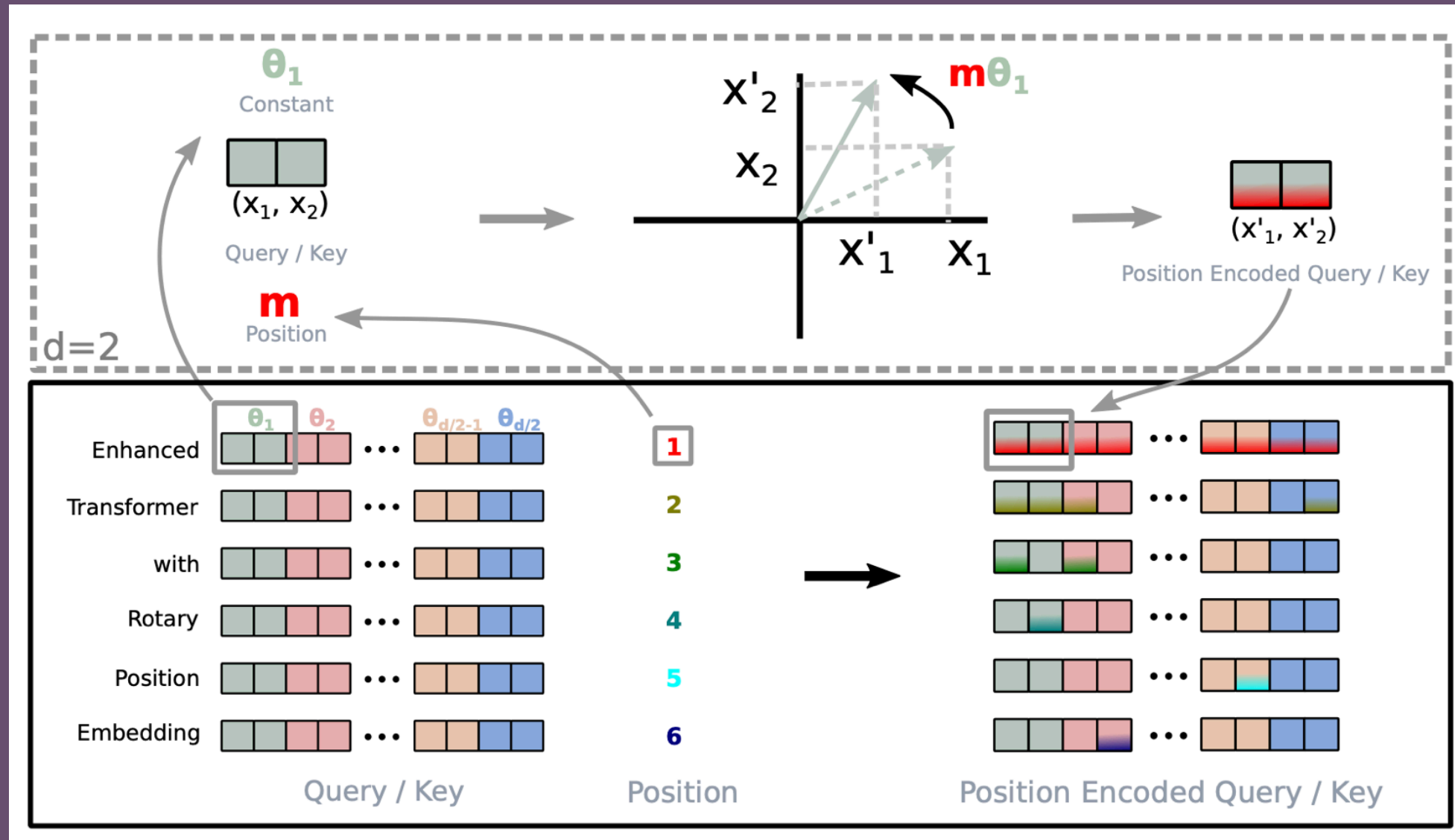
What are standard position embeddings

Two embedding tables, one for tokens (traditionally called word embeddings) and one for their positions.

Then, initial token representation is as follows:

$$h_i^{(0)} = \text{word embed}(x_i) + \text{pos embed}(i)$$

Rotary Position Embeddings (RopE)¹



1. Su et al (2021) ↩

RoPE intuition

- Multiply representations with a rotation matrix based on their position
- Can be integrated into the attention formulation (not only on input level as standard position embeddings)
- Position information doesn't need to be passed through & retained in all blocks, but is ingested into the attention formulation in each transformer block

3.2.2 General form

In order to generalize our results in 2D to any $\mathbf{x}_i \in \mathbb{R}^d$ where d is even, we divide the d -dimension space into $d/2$ sub-spaces and combine them in the merit of the linearity of the inner product, turning $f_{\{q,k\}}$ into:

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m \quad (14)$$

where

$$\mathbf{R}_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix} \quad (15)$$

is the rotary matrix with pre-defined parameters $\Theta = \{\theta_i = 10000^{-2(i-1)/d}, i \in [1, 2, \dots, d/2]\}$. A graphic illustration of RoPE is shown in Figure (1). Applying our RoPE to self-attention in Equation (2), we obtain:

$$\mathbf{q}_m^\top \mathbf{k}_n = (\mathbf{R}_{\Theta, m}^d \mathbf{W}_q \mathbf{x}_m)^\top (\mathbf{R}_{\Theta, n}^d \mathbf{W}_k \mathbf{x}_n) = \mathbf{x}^\top \mathbf{W}_q \mathbf{R}_{\Theta, n-m}^d \mathbf{W}_k \mathbf{x}_n \quad (16)$$

Multi-head Latent Attention

Revisit standard multi-head attention formulation

Queries, Keys, and Values come from a linear projection from current token representation

$$q = W_q h, k = W_k h, v = W_v h$$

...and are then sliced into n heads $[q_1, q_2, \dots, q_n] = q$

where each q_i is of size $\frac{d_{\text{key}}}{n}$.

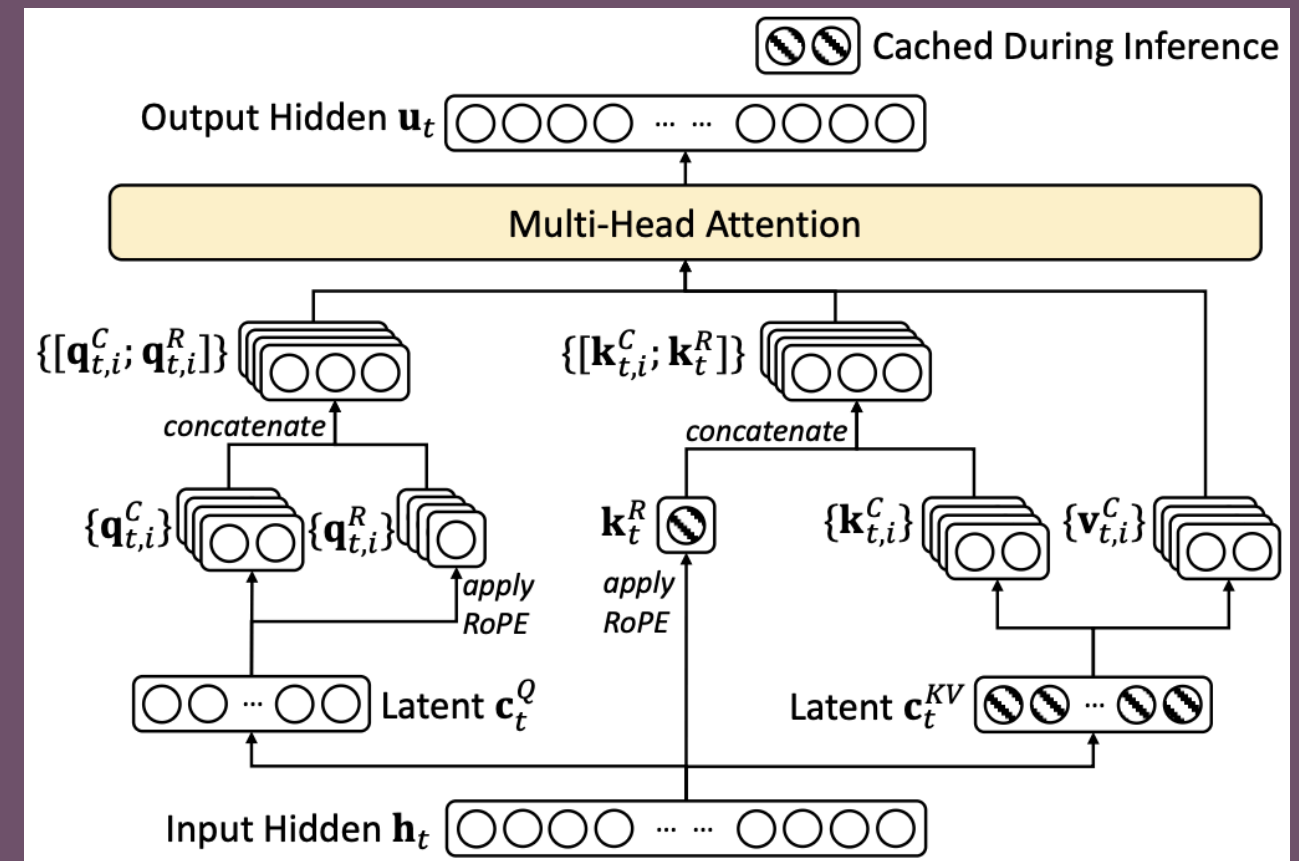
Then we calculate similarity between all pairs of keys and queries and aggregate values accordingly

$$\text{softmax}\left(\frac{QK^T}{d_{\text{key}}}\right)V$$

Multi-head Latent Attention

Low-rank joint compression of keys and values

Significantly reduced KV-cache during generation



Roadmap to GLM 5

- ☒ Mixture of Experts
- ☒ DeepSeek Sparse Attention
- ☒ Multi-latent Attention
 - ☒ Rotary Position Embeddings
- ☐ Multi-token Prediction
- ☐ Muon Optimizer

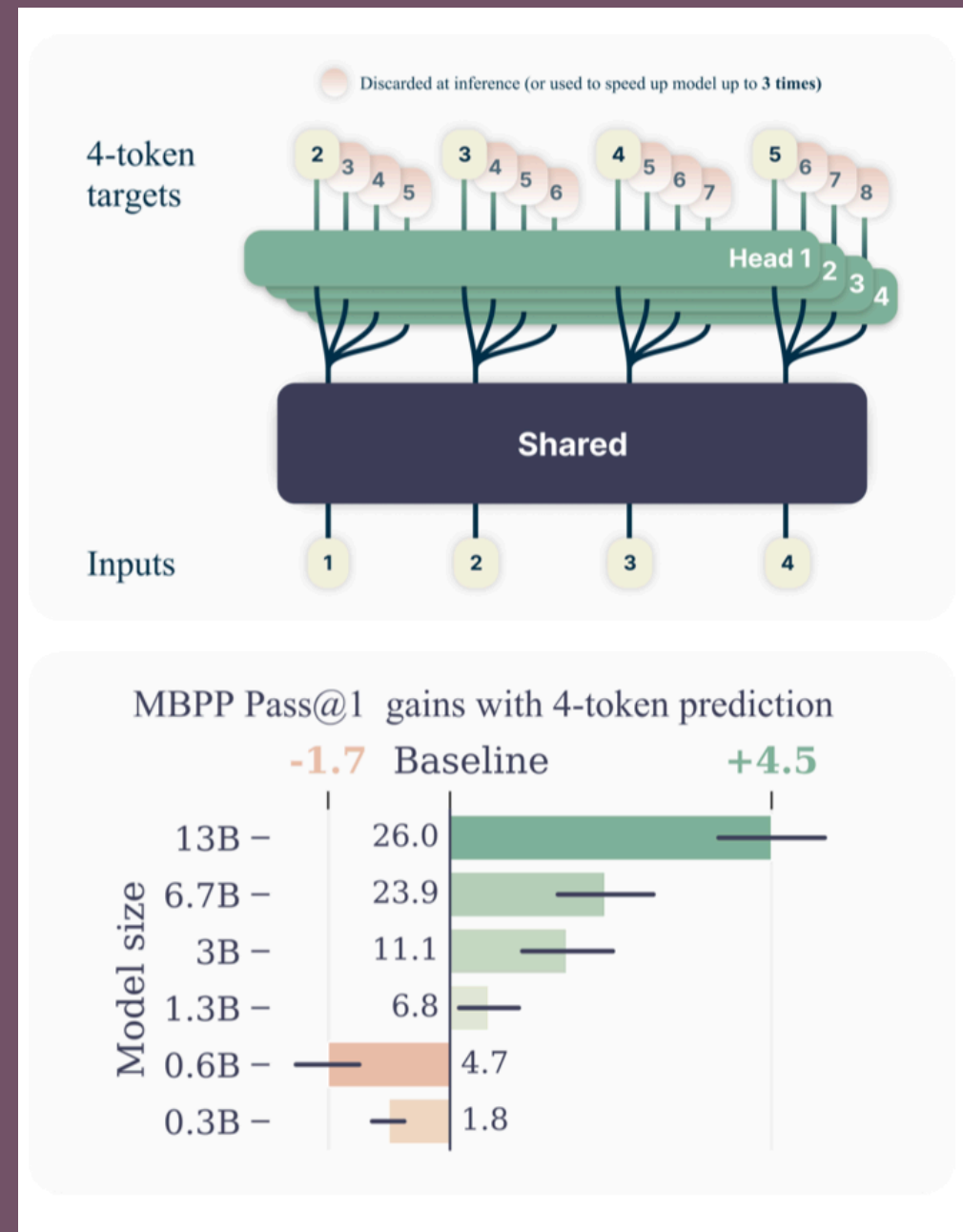
Multi-Token prediction

Multi-token prediction

Key Idea Instead of predicting only the next token, predict the next handful of tokens -- each with a separate language modeling head.

Rationale This provides a better training signal. Separate language modeling heads can be discarded after training.

Extension of GLM 5: Share parameters between language modeling heads.



Roadmap to GLM 5

- ☒ Mixture of Experts
- ☒ DeepSeek Sparse Attention
- ☒ Multi-latent Attention
 - ☒ Rotary Position Embeddings
- ☒ Multi-token Prediction
- ☐ Muon Optimizer

AdamW optimizer

- Adam = Adaptive moment estimation
- Stochastic gradient descent with momentum
- **Plus:** also keep track of the variance of updates
 - Big updates for rarely touched parameters
 - Small updates for often touched parameters
- **Great for overparameterized models**
- The W part: Fixed weight decay

Muon = Momentum Orthogonalized by Newton-Schulz¹

1. Do stochastic gradient descent w/ momentum
2. Postprocess update via Newton-Schulz

This second step approximately orthogonalizes the update matrix

- Big updates for rarely touched parameters
- Small updates for often touched parameters

Why Muon then? More token efficient

1. Blog post on Muon ↩

Algorithm 1 MuonClip Optimizer

```
1: for each training step  $t$  do
2:   // 1. Muon optimizer step
3:   for each weight  $\mathbf{W} \in \mathbb{R}^{n \times m}$  do
4:      $\mathbf{M}_t = \mu \mathbf{M}_{t-1} + \mathbf{G}_t$  ▷  $\mathbf{M}_0 = \mathbf{0}$ ,  $\mathbf{G}_t$  is the grad of  $\mathbf{W}_t$ ,  $\mu$  is momentum
5:      $\mathbf{O}_t = \text{Newton-Schulz}(\mathbf{M}_t) \cdot \sqrt{\max(n, m)} \cdot 0.2$  ▷ Match Adam RMS
6:      $\mathbf{W}_t = \mathbf{W}_{t-1} - \eta(\mathbf{O}_t + \lambda \mathbf{W}_{t-1})$  ▷ learning rate  $\eta$ , weight decay  $\lambda$ 
7:   end for
8:   // 2. QK-Clip
9:   for each attention head  $h$  in every attention layer of the model do
10:    Obtain  $S_{\max}^h$  already computed during forward
11:    if  $S_{\max}^h > \tau$  then
12:       $\gamma \leftarrow \tau / S_{\max}^h$ 
13:       $\mathbf{W}_{qc}^h \leftarrow \mathbf{W}_{qc}^h \cdot \sqrt{\gamma}$ 
14:       $\mathbf{W}_{kc}^h \leftarrow \mathbf{W}_{kc}^h \cdot \sqrt{\gamma}$ 
15:       $\mathbf{W}_{qr}^h \leftarrow \mathbf{W}_{qr}^h \cdot \gamma$ 
16:    end if
17:  end for
18: end for
```

Some Caveats

- Muon excels at token efficiency
- But can be a bit unstable
- Therefore, every other new LLM paper proposes some small variation to Muon
- Also; Muon is only used for feedforward layer parameters. Embeddings still use AdamW.

Roadmap to GLM 5

- ☒ Mixture of Experts
- ☒ DeepSeek Sparse Attention
- ☒ Multi-latent Attention
 - ☒ Rotary Position Embeddings
- ☒ Multi-token Prediction
- ☒ Muon Optimizer
- ☐ Modern basics (see [code for GLM 5 family](#))
 - ☐ RMSNorm
 - ☐ SwiGLU

RMSNorm

What was LayerNorm again?

A regularization method to normalize intermediate representation.

$$y = \frac{x - E[x]}{\sqrt{\text{Var}[X] + \epsilon}} * \gamma + \beta$$

where γ and β are learnable parameters.

Intuition: Normalizes to zero mean unit variance (and then rescales as needed with γ and β).

Now what is RMSNorm?

RMSNorm = Root mean squared normalization

$$y_i = \frac{x_i}{\text{RMS}(x)} * \gamma_i$$

$$\text{where } \text{RMS}(x) = \sqrt{\epsilon + \frac{1}{n} \sum_{i=1}^n x_i^2}$$

and γ are (optional) learnable parameters.

Intuition: Makes assumption of zero mean and only normalizes variance. **A bit cheaper to compute.**

SwiGLU

Activation functions

Dead ReLU's and how to do even better: GELU

- Most used activation function in Transformers is the Gaussian Error Linear GELU
- Exceeds performance of ReLU and ELU

Idea: GELU scales activations by how much larger they are than other activations, assuming $N(0,1)$ distribution

$$\text{GELU}(x) = xP(X \leq x) = x\Phi(x) = x \cdot \frac{1}{2} \left[1 + \text{erf}(x/\sqrt{2}) \right]$$

We can approximate the GELU with

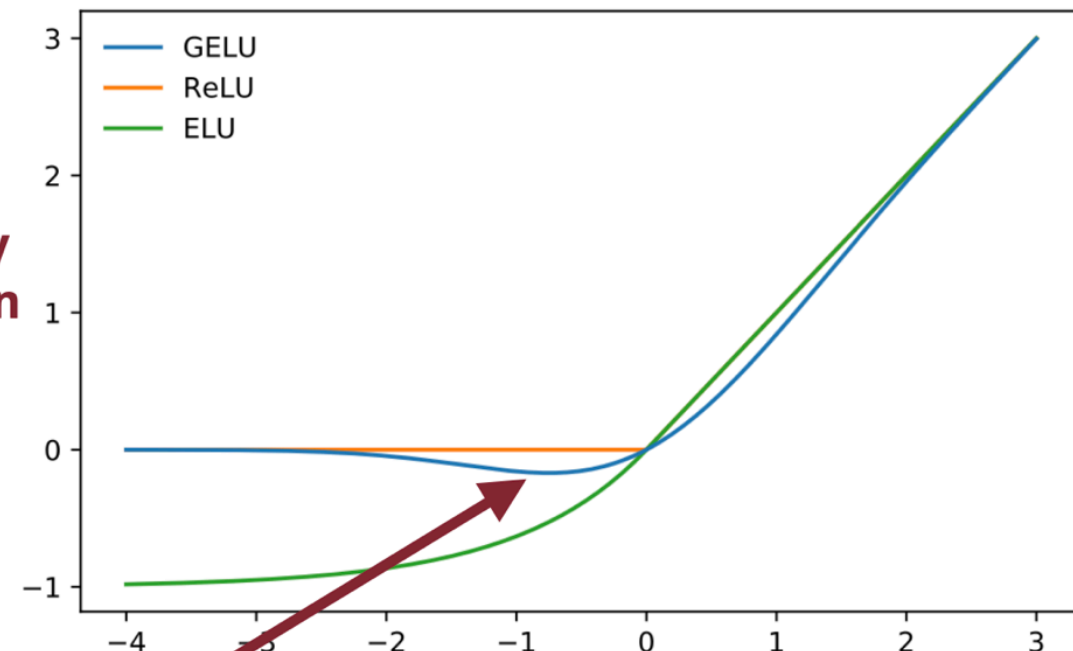
$$0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$$

or

$$x\sigma(1.702x),$$

**Pattern also seen in other recent activation functions:
x multiplied by logistic sigmoid of x**

From arXiv:1606.08415v5



Leads to a smooth “bump” just before zero

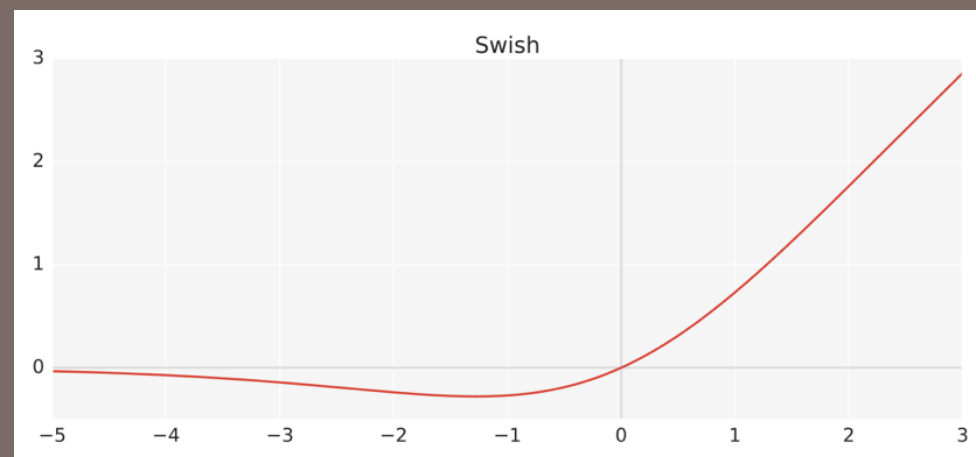
From GELU to Swish¹/SiLU

$$\text{Swish}_{\beta}(x) = \text{sigmoid}(\beta x) \cdot x$$

β is a hyperparameter, usually = 1

Similar to GELU, but easier to compute

Sometimes called self-gating unit because the sigmoid involved



1. Ramachandran, Zoph, & Le (2017) ↩

Gated Linear Units (GLUs) in general

Two transformations, one sigmoid activated acts as a **gate** to the other

$$h = \text{sigmoid}(xW) * (xV)$$

SwiGLU¹

SwiGLU: a mix of Swish and GLU

$$h = \text{Swish}_{\beta}(xW) * (xV)$$

A feedforward layer with SwiGLU:

$$\text{FFN}(x) = (\text{Swish}_{\beta}(xW_1) * (xV))W_2$$

1. Shazeer (2020) ↩

Roadmap to GLM 5

- ✓ Mixture of Experts
- ✓ DeepSeek Sparse Attention
- ✓ Multi-latent Attention
 - ✓ Rotary Position Embeddings
- ✓ Multi-token Prediction
- ✓ Muon Optimizer
- ✓ Modern basics
 - ✓ RMSNorm
 - ✓ SwiGLU

Keywords

- Mixture of Experts
- DeepSeek Sparse Attention
- Multi-Latent Attention
- Rotary Position Embeddings
- Multi-token Prediction
- Muon Optimizer
- RMSNorm
- Gated linear units

How's my teaching?

